00b — Setup & Data Preparation (FMA Small)

Prepara il dataset FMA Small (8 generi). Usa stessi parametri audio di GTZAN (sr=22050, n_mels=128, durata segmenti ~3s).

Panoramica

Questo notebook prepara FMA Small in 5 step chiari:

1. Configurazione e logging (parametri, caps, parallelismo)
2. Download/Verifica integrità (mirror ufficiale + check decodifica)
3. Preparazione/Feature extraction (ffmpeg, segmenti ~3s, padding robusto)
4. Bilanciamento per classe e split riproducibili
5. Salvataggio array + report riassuntivo

Suggerimenti:

- Per corse veloci: riduci MAX_TRACKS_PER_CLASS e/o FFMPEG_MAX_SECS.
- Per output puliti: tutti i messaggi usano il logger con timestamp.

In [1]:
```
# Paths & placeholders
import os, sys, json, pickle, numpy as np, subprocess, shutil, re, zipfile
from pathlib import Path
PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
FMA_ROOT = PROJECT_ROOT/'data'/'fma_small'
PROCESSED = PROJECT_ROOT/'data'/'processed_fma'
KAGGLE_DIR = PROJECT_ROOT/'kaggle'
PROCESSED.mkdir(parents=True, exist_ok=True)
print('FMA_ROOT:', FMA_ROOT)
print('PROCESSED:', PROCESSED)

# User options
ONLY_SMALL = True   # True => scarica solo la parte "small" (consigliato per
PREFER_KAGGLEHUB = False  # Se True, prova KaggleHub prima (potrebbe scarica

# Official mirror URLs (FMA project)
OFFICIAL_SMALL_URL = 'https://os.unil.cloud.switch.ch/fma/fma_small.zip'

# Helpers

def ensure_package(pkg_spec: str):
    try:
        __import__(pkg_spec.split('[')[0])
        return True
    except Exception:
        try:
```

```
            print(f'Installing {pkg_spec} ...')
            subprocess.run([sys.executable, '-m', 'pip', 'install', pkg_spec
            return True
        except Exception as e:
            print(f'Failed to install {pkg_spec}:', e)
            return False

# Prefer ffmpeg-backed decoding via imageio-ffmpeg (bundled binary)
try:
    if ensure_package('imageio-ffmpeg'):
        import imageio_ffmpeg
        ffexe = imageio_ffmpeg.get_ffmpeg_exe()
        ffdir = str(Path(ffexe).parent)
        os.environ['PATH'] = ffdir + os.pathsep + os.environ.get('PATH', '')
        os.environ.setdefault('AUDIOREAD_BACKEND', 'ffmpeg')
        os.environ.setdefault('AUDIOREAD_PLUGIN', 'ffmpeg')
        print('FFmpeg configured from imageio-ffmpeg at:', ffexe)
except Exception as e:
    print('FFmpeg setup warning:', e)
```

```
FMA_ROOT: /home/alepot55/Desktop/projects/naml_project/data/fma_small
PROCESSED: /home/alepot55/Desktop/projects/naml_project/data/processed_fma
Installing imageio-ffmpeg ...
Requirement already satisfied: imageio-ffmpeg in /home/alepot55/Desktop/proj
ects/naml_project/venv/lib/python3.12/site-packages (0.6.0)
FFmpeg configured from imageio-ffmpeg at: /home/alepot55/Desktop/projects/na
ml_project/venv/lib/python3.12/site-packages/imageio_ffmpeg/binaries/ffmpeg-
linux-x86_64-v7.0.2
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: pip install --upgrade pip
```

In [ ]:

```
# Optional: Indian Music Genre dataset acquisition via KaggleHub
import os, shutil
from pathlib import Path
import kagglehub



INDIAN_DATASET_ID = os.environ.get('INDIAN_DATASET_ID', 'winchester19/indiar
INDIAN_ROOT = PROJECT_ROOT / 'data' / 'indian_music'


def ensure_indian_dataset():
    INDIAN_ROOT.mkdir(parents=True, exist_ok=True)
    log(f'Trying KaggleHub dataset_download({INDIAN_DATASET_ID})')
    try:
        path = kagglehub.dataset_download(INDIAN_DATASET_ID)
        log(f'KaggleHub path: {path}')
        # Copy/rsync-like: move audio folders into INDIAN_ROOT (idempotent)
        src = Path(path)
        # The dataset has top-level genre folders; copy them if not present
        copied = 0
        for item in src.iterdir():
            if item.is_dir():
                dest = INDIAN_ROOT / item.name
                if not dest.exists():
```

```
                    shutil.copytree(item, dest)
                    copied += 1
        log(f'Indian dataset prepared at {INDIAN_ROOT} (copied {copied} dirs
        return INDIAN_ROOT
    except Exception as e:
        log(f'Indian dataset download failed: {e}', level='WARN')
        return None


# Uncomment to fetch when needed:


# ensure_indian_dataset()
```

In [2]:
```
# Logging utility for clean, consistent messages
import os
from datetime import datetime

VERBOSE = os.environ.get('FMA_VERBOSE', '1') == '1'

def log(msg: str, level: str = 'INFO'):
    """Print a timestamped log line. Set FMA_VERBOSE=0 to reduce noise."""
    if not VERBOSE and level == 'INFO':
        return
    ts = datetime.now().strftime('%H:%M:%S')
    print(f'[{ts}] {level}: {msg}')
```

In [3]:
```
# Configuration — fast, reproducible, and controllable
import os
from multiprocessing import cpu_count

# Audio params (mirror GTZAN defaults)
SR = int(os.environ.get('FMA_SR', '22050'))
N_MELS = int(os.environ.get('FMA_N_MELS', '128'))
HOP = int(os.environ.get('FMA_HOP', '512'))
SEG_DUR = float(os.environ.get('FMA_SEG_DUR', '2.97'))  # seconds
N_SEGMENTS = int(os.environ.get('FMA_N_SEGMENTS', '10'))
FFMPEG_MAX_SECS = float(os.environ.get('FMA_FFMPEG_MAX_SECS', '30'))  # deco

# Decoding backend: 'auto' (try librosa, fallback ffmpeg), 'librosa', or 'ff
DECODE_BACKEND = os.environ.get('FMA_DECODE_BACKEND', 'auto').lower()

# Speed/size controls
MAX_TRACKS_PER_CLASS = int(os.environ.get('FMA_MAX_TRACKS_PER_CLASS', '40'))
PARALLEL = os.environ.get('FMA_PARALLEL', '0') == '1'  # set 1 to enable job
N_JOBS = int(os.environ.get('FMA_PARALLEL_JOBS', str(max(1, cpu_count() // 2

# Cache behavior
FORCE_REDO = os.environ.get('FMA_FORCE_REDO', '0') == '1'  # if 1, ignore ca
TRUST_CACHE = os.environ.get('FMA_TRUST_CACHE', '1') == '1'  # if 1, trust m

# Derived target time frames (≈130 for 2.97s at hop=512, sr=22050)
T_TARGET = int(round(SEG_DUR * SR / HOP)) + 1

CONFIG = {
```

```
    'SR': SR,
    'N_MELS': N_MELS,
    'HOP': HOP,
    'SEG_DUR': SEG_DUR,
    'N_SEGMENTS': N_SEGMENTS,
    'FFMPEG_MAX_SECS': FFMPEG_MAX_SECS,
    'DECODE_BACKEND': DECODE_BACKEND,
    'MAX_TRACKS_PER_CLASS': MAX_TRACKS_PER_CLASS,
    'PARALLEL': PARALLEL,
    'N_JOBS': N_JOBS,
    'T_TARGET': T_TARGET,
}

print('Config: SR=', SR, 'N_MELS=', N_MELS, 'HOP=', HOP, 'SEG_DUR=', SEG_DUR
print('Caps/Speed: MAX_TRACKS_PER_CLASS=', MAX_TRACKS_PER_CLASS, 'FFMPEG_MAX
print('Derived: T_TARGET=', T_TARGET)
```

```
Config: SR= 22050 N_MELS= 128 HOP= 512 SEG_DUR= 2.97 N_SEGMENTS= 10
Caps/Speed: MAX_TRACKS_PER_CLASS= 40 FFMPEG_MAX_SECS= 30.0 DECODE_BACKEND= a
uto PARALLEL= False N_JOBS= 10
Derived: T_TARGET= 129
```

In [4]:

```
# Download logic (KaggleHub-first; fall back to official mirror; idempotent)
import random, time, shutil
from pathlib import Path

# Ensure ffmpeg-backed audioread here too (defensive, in case cell 1 not run
os.environ.setdefault('AUDIOREAD_BACKEND', 'ffmpeg')
os.environ.setdefault('AUDIOREAD_PLUGIN', 'ffmpeg')

# Self-contained helpers
import urllib.request

def http_download(url: str, dest: Path, chunk_size: int = 1 << 20) -> bool:
    try:
        dest.parent.mkdir(parents=True, exist_ok=True)
        with urllib.request.urlopen(url) as r, open(dest, 'wb') as f:
            while True:
                chunk = r.read(chunk_size)
                if not chunk:
                    break
                f.write(chunk)
        log(f'Downloaded {url} -> {dest}')
        return True
    except Exception as e:
        log(f'HTTP download failed: {e}', level='WARN')
        return False

import zipfile

def unzip_to_dir(zip_path: Path, out_dir: Path) -> bool:
    try:
        with zipfile.ZipFile(zip_path, 'r') as zf:
            zf.extractall(out_dir)
        log(f'Unzipped {zip_path} -> {out_dir}')
        return True
```

```
    except Exception as e:
        log(f'Unzip failed: {e}', level='WARN')
        return False

# Lightweight ffmpeg probe to validate decode-ability
import subprocess as sp

def ffmpeg_probe_ok(path: Path) -> bool:
    try:
        r = sp.run(['ffmpeg', '-v', 'error', '-i', str(path), '-f', 'null',
        return r.returncode == 0
    except Exception:
        return False

import librosa

def quick_decode_check(mp3_paths, sample_size=200, sr=22050, secs=1.0):
    mp3_list = list(mp3_paths)
    if not mp3_list:
        return 0, 0
    k = min(sample_size, len(mp3_list))
    sampled = random.sample(mp3_list, k)
    ok = 0
    for p in sampled:
        try:
            y, _ = librosa.load(str(p), sr=sr, mono=True, duration=secs, res
            if y is not None and y.size > 0:
                ok += 1
        except Exception:
            continue
    return ok, k

# Paths
dl_dir = PROJECT_ROOT/'data'/'fma_download'
dl_dir.mkdir(parents=True, exist_ok=True)

# Consider dataset present only if there are mp3 files available
mp3_present = FMA_ROOT.exists() and any(FMA_ROOT.rglob('*.mp3'))
needs_redownload = False

if mp3_present:
    mp3_files_all = list(FMA_ROOT.rglob('*.mp3'))
    ok, k = quick_decode_check(mp3_files_all, sample_size=120)
    ratio = (ok / k) if k else 0.0
    log(f"Decode check sample: {ok}/{k} ok ({ratio*100:.1f}%)")
    if ratio < 0.5:
        log('High fraction of decode failures, considering re-download...',
        needs_redownload = True
else:
    needs_redownload = True

used_download = False
small_dir = None

# Prefer KaggleHub (requested). Controlled by FMA_KAGGLEHUB=1 (default 1).
USE_KAGGLEHUB = os.environ.get('FMA_KAGGLEHUB', '1') == '1'
```

```
KAGGLEHUB_DATASET_ID = 'imsparsh/fma-free-music-archive-small-medium'

if needs_redownload and USE_KAGGLEHUB:
    try:
        try:
            import kagglehub  # type: ignore
        except Exception:
            if 'ensure_package' in globals():
                ensure_package('kagglehub')
                import kagglehub  # type: ignore
        base_path = Path(kagglehub.dataset_download(KAGGLEHUB_DATASET_ID))
        log(f'KaggleHub cache: {base_path}')
        # Locate fma_small inside the dataset cache
        cand_dirs = [p for p in base_path.rglob('fma_small') if p.is_dir()]
        if not cand_dirs:
            zips = list(base_path.rglob('fma_small*.zip'))
            if zips:
                unzip_to_dir(zips[0], zips[0].parent)
                cand_dirs = [p for p in base_path.rglob('fma_small') if p.is
        if cand_dirs:
            small_dir = cand_dirs[0]
            log(f'Found fma_small in KaggleHub cache: {small_dir}')
            if FMA_ROOT.exists():
                backup = FMA_ROOT.with_name(f"fma_small_backup_{int(time.tim
                shutil.move(str(FMA_ROOT), str(backup))
                log(f'Previous fma_small backed up to {backup}', level='INFO
            FMA_ROOT.mkdir(parents=True, exist_ok=True)
            for item in Path(small_dir).iterdir():
                target = FMA_ROOT/item.name
                if item.is_dir():
                    shutil.copytree(item, target, dirs_exist_ok=True)
                else:
                    shutil.copy2(item, target)
            log(f'FMA small ready at {FMA_ROOT} (from KaggleHub)')
            used_download = True
        else:
            log('KaggleHub: fma_small not found inside dataset cache.', leve
    except Exception as e:
        log(f'KaggleHub download failed: {e}', level='WARN')

# Fallback to official mirror if KaggleHub not used or failed
if needs_redownload and not used_download:
    local_zip = dl_dir/'fma_small.zip'
    if local_zip.exists():
        log(f'Found existing archive: {local_zip} — reusing')
    else:
        log(f'Trying official FMA mirror: {OFFICIAL_SMALL_URL}')
        if not http_download(OFFICIAL_SMALL_URL, local_zip):
            log('Mirror download failed; cannot proceed automatically.', lev
    if local_zip.exists():
        if unzip_to_dir(local_zip, dl_dir):
            cand_dirs = list(dl_dir.rglob('fma_small'))
            small_dir = cand_dirs[0] if cand_dirs else None
        if small_dir and Path(small_dir).exists():
            if FMA_ROOT.exists():
                backup = FMA_ROOT.with_name(f"fma_small_backup_{int(time.tim
```

```
                shutil.move(str(FMA_ROOT), str(backup))
                log(f'Previous fma_small backed up to {backup}', level='INFO
            FMA_ROOT.mkdir(parents=True, exist_ok=True)
            for item in Path(small_dir).iterdir():
                target = FMA_ROOT/item.name
                if item.is_dir():
                    shutil.copytree(item, target, dirs_exist_ok=True)
                else:
                    shutil.copy2(item, target)
            log(f'FMA small ready at {FMA_ROOT}')
            used_download = True
        else:
            log('Unzip complete but fma_small folder not found in archive.',

if not used_download and mp3_present and not needs_redownload:
    log('FMA small present and decode check passed — skipping download.')

# Post-check a tiny sample to diagnose failures without re-downloading again
try:
    sample_files = list(FMA_ROOT.rglob('*.mp3'))
    if sample_files:
        sample_files = random.sample(sample_files, min(10, len(sample_files)
        sizes = [Path(p).stat().st_size for p in sample_files]
        probes = [ffmpeg_probe_ok(Path(p)) for p in sample_files]
        log(f'Sample file sizes (bytes): {sizes}')
        log(f'ffmpeg probe ok count: {sum(probes)}/{len(probes)}')
        if sum(probes) == 0:
            log('ffmpeg cannot decode sampled files. Archive may be corrupte
except Exception as e:
    log(f'Post-check diagnostics failed: {e}', level='WARN')
```

```
[17:41:47] INFO: Decode check sample: 0/120 ok (0.0%)
[17:41:47] WARN: High fraction of decode failures, considering re-downloa
d...
Download already complete (31961630661 bytes).
Extracting files...
[17:45:50] INFO: KaggleHub cache: /home/alepot55/.cache/kagglehub/datasets/i
msparsh/fma-free-music-archive-small-medium/versions/1
[17:45:50] INFO: Found fma_small in KaggleHub cache: /home/alepot55/.cache/k
agglehub/datasets/imsparsh/fma-free-music-archive-small-medium/versions/1/fm
a_small
[17:45:50] INFO: Previous fma_small backed up to /home/alepot55/Desktop/proj
ects/naml_project/data/fma_small_backup_1756050350
[17:46:09] INFO: FMA small ready at /home/alepot55/Desktop/projects/naml_pro
ject/data/fma_small (from KaggleHub)
[17:46:10] INFO: Sample file sizes (bytes): [962041, 1202384, 480908, 120228
6, 961286, 962035, 720635, 1201253, 1201198, 1202123]
[17:46:10] INFO: ffmpeg probe ok count: 10/10
```

> Nota: per mantenere il progetto offline-safe, questo notebook non scarica automaticamente FMA. Posiziona `fma_small` in `data/fma_small`.

In [5]:

```
# Processing — metadata mapping, balanced split, caching, and extraction
import os
import re
import json
import pickle
from pathlib import Path
import numpy as np
import pandas as pd
import librosa
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tqdm import tqdm
import warnings
from contextlib import contextmanager, redirect_stderr, redirect_stdout

# Prefer ffmpeg backend for audioread if available
os.environ.setdefault("AUDIOREAD_PLUGIN", "ffmpeg")
os.environ.setdefault("AUDIOREAD_BACKEND", "ffmpeg")

@contextmanager
def quiet_audioio():
    """Suppress decoder stdout/stderr and warnings (mpg123/ffmpeg chatter)."
    with open(os.devnull, 'w') as devnull:
        with redirect_stderr(devnull), redirect_stdout(devnull):
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                yield

# Cache metadata file to avoid recomputation
META_JSON = PROCESSED/'fma_prep_meta.json'
REQUIRED_FILES = [PROCESSED/f for f in ['X_train.npy','y_train.npy','X_val.r

# Early exit if cache exists and matches current config
if not FORCE_REDO and all(p.exists() for p in REQUIRED_FILES) and META_JSON.
    try:
        with open(META_JSON,'r') as f:
            meta = json.load(f)
        cfg = meta.get('config', {})
        cfg_match = all(str(cfg.get(k)) == str(v) for k, v in CONFIG.items()
        if TRUST_CACHE and cfg_match:
            log('Processed FMA artifacts already present and config matches.
            log(f"X shapes: {np.load(PROCESSED/'X_train.npy').shape}, {np.lo
            raise SystemExit
        else:
            log('Cache exists but config differs (or TRUST_CACHE=0). Proceed
    except Exception:
        log('Cache check failed; proceeding to compute.', level='WARN')

# Ensure dataset presence
if not FMA_ROOT.exists():
    raise FileNotFoundError(
        'Dataset FMA Small non trovato. Esegui la prima cella (download) opp
    )

# Ensure metadata (tracks.csv)
```

```
METADATA_ROOT = PROJECT_ROOT/'data'/'fma_metadata'
TRACKS_CSV = METADATA_ROOT/'tracks.csv'
OFFICIAL_METADATA_URL = 'https://os.unil.cloud.switch.ch/fma/fma_metadata.zi
dl_dir = PROJECT_ROOT/'data'/'fma_download'
dl_dir.mkdir(parents=True, exist_ok=True)

if not TRACKS_CSV.exists():
    cand = list(dl_dir.rglob('tracks.csv')) if dl_dir.exists() else []
    if cand:
        METADATA_ROOT = cand[0].parent
        TRACKS_CSV = cand[0]
    else:
        log('tracks.csv non trovato. Provo a scaricare fma_metadata.zip (mir
        meta_zip = dl_dir/'fma_metadata.zip'
        if not meta_zip.exists():
            if 'http_download' in globals():
                http_download(OFFICIAL_METADATA_URL, meta_zip)
        if meta_zip.exists():
            if 'unzip_to_dir' in globals():
                unzip_to_dir(meta_zip, dl_dir)
            cand = list(dl_dir.rglob('tracks.csv'))
            if cand:
                METADATA_ROOT = cand[0].parent
                TRACKS_CSV = cand[0]

if not TRACKS_CSV.exists():
    raise FileNotFoundError('tracks.csv non disponibile. Scarica fma_metadat

log(f'Usando metadata: {TRACKS_CSV}')

# Load metadata; tracks.csv uses MultiIndex columns
tracks = pd.read_csv(TRACKS_CSV, header=[0, 1], index_col=0, low_memory=Fals
if ('track', 'genre_top') not in tracks.columns:
    raise RuntimeError('Colonna (track, genre_top) non trovata in tracks.csv

id_to_genre = tracks[('track', 'genre_top')]

# Gather mp3 files and map to genre_top via track id (filename stem)
mp3_files = sorted([p for p in FMA_ROOT.rglob('*.mp3')])
if len(mp3_files) == 0:
    raise RuntimeError('Nessun file .mp3 trovato in fma_small.')

files, labels = [], []
for p in mp3_files:
    try:
        tid = int(p.stem)
    except Exception:
        continue
    lab = id_to_genre.get(tid)
    if pd.isna(lab):
        continue
    files.append(str(p))
    labels.append(str(lab))

if len(files) == 0:
    raise RuntimeError('Nessun file etichettato trovato: controlla che track
```

```
# Balanced sampling per class (optional speed-up)
from collections import defaultdict
pairs = list(zip(files, labels))
by_class = defaultdict(list)
for f, y in pairs:
    by_class[y].append(f)

if MAX_TRACKS_PER_CLASS > 0:
    import random
    capped_files, capped_labels = [], []
    rng = random.Random(42)
    for y, lst in by_class.items():
        lst_sorted = sorted(lst)
        rng.shuffle(lst_sorted)
        take = lst_sorted[:MAX_TRACKS_PER_CLASS]
        capped_files.extend(take)
        capped_labels.extend([y] * len(take))
    log(f'Applied cap per class: {MAX_TRACKS_PER_CLASS}')
    files, labels = capped_files, capped_labels
else:
    log('No per-class cap applied.')

unique_classes = sorted(set(labels))
from collections import Counter
cnt = Counter(labels)
can_stratify = len(unique_classes) >= 2 and min(cnt.values()) >= 2
log(f"Classes: {unique_classes} | Total tracks considered: {len(labels)}")
log(f'Stratify enabled: {can_stratify}')

strat_arg = labels if can_stratify else None
if not can_stratify:
    log('Stratify disattivato (classi insufficienti o troppo sbilanciate). U

train_val_files, test_files, train_val_labels, test_labels = train_test_spli
    files, labels, test_size=0.2, random_state=42, stratify=strat_arg
)

strat_arg_tv = train_val_labels if can_stratify else None
train_files, val_files, train_labels, val_labels = train_test_split(
    train_val_files, train_val_labels, test_size=0.25, random_state=42, stra
)

log(f'Train/Val/Test sizes: {len(train_files)}/{len(val_files)}/{len(test_fi

# Decoders
import subprocess as sp

def load_audio_librosa(fp: str, sr: int, max_secs: float):
    with quiet_audioio():
        y, _ = librosa.load(fp, sr=sr, mono=True, duration=max_secs, res_typ
    return y

def load_audio_ffmpeg(fp: str, sr: int, max_secs: float):
    try:
        # Decode to raw PCM via ffmpeg and read as float32
```

```
        cmd = ['ffmpeg', '-v', 'error', '-i', fp, '-f', 'f32le', '-acodec',
        p = sp.run(cmd, stdout=sp.PIPE, stderr=sp.PIPE, check=False)
        if p.returncode != 0 or not p.stdout:
            return None
        y = np.frombuffer(p.stdout, dtype=np.float32)
        return y
    except Exception:
        return None

def load_audio(fp: str, sr: int, max_secs: float):
    if DECODE_BACKEND == 'librosa':
        return load_audio_librosa(fp, sr, max_secs)
    if DECODE_BACKEND == 'ffmpeg':
        return load_audio_ffmpeg(fp, sr, max_secs)
    # auto: try librosa then fallback to ffmpeg
    y = None
    try:
        y = load_audio_librosa(fp, sr, max_secs)
    except Exception:
        y = None
    if y is None or (isinstance(y, np.ndarray) and y.size == 0):
        y = load_audio_ffmpeg(fp, sr, max_secs)
    return y

# Feature extraction helpers
def extract_one(args):
    fp, lab = args
    try:
        ysig = load_audio(fp, SR, FFMPEG_MAX_SECS)
        if ysig is None or len(ysig) == 0:
            return None
        seg_len = int(SR * SEG_DUR)
        out = []
        total = len(ysig)
        for s in range(N_SEGMENTS):
            st, en = s * seg_len, s * seg_len + seg_len
            if st >= total:
                break
            seg_sig = ysig[st:en]
            if len(seg_sig) < seg_len:
                pad = np.zeros(seg_len - len(seg_sig), dtype=seg_sig.dtype)
                seg_sig = np.concatenate([seg_sig, pad])
            mel = librosa.feature.melspectrogram(y=seg_sig, sr=SR, n_mels=N_
            out.append(librosa.power_to_db(mel, ref=np.max))
        return (out, [lab] * len(out)) if out else None
    except Exception:
        return None

# Extraction driver (parallel optional)
def extract(files, labels):
    from itertools import chain
    pairs = list(zip(files, labels))
    results = []
    if PARALLEL and len(pairs) > 1:
        try:
            from joblib import Parallel, delayed
```

```
            results = Parallel(n_jobs=N_JOBS, prefer='threads')(delayed(extr
        except Exception as e:
            log(f'Parallel extract fallback to serial due to: {e}', level='W
            results = [extract_one(p) for p in tqdm(pairs)]
    else:
        results = [extract_one(p) for p in tqdm(pairs)]

    X, y = [], []
    for res in results:
        if res is None:
            continue
        out, labs = res
        X.extend(out)
        y.extend(labs)
    return X, y

Xtr_list, ytr_txt = extract(train_files, train_labels)
Xva_list, yva_txt = extract(val_files, val_labels)
Xte_list, yte_txt = extract(test_files, test_labels)

# Unify time dimension to T=T_TARGET (pad/crop)
def unify(lst, T=T_TARGET):
    out = []
    for s in lst:
        if s.shape[1] > T:
            out.append(s[:, :T])
        else:
            out.append(np.pad(s, ((0, 0), (0, T - s.shape[1])), 'constant'))
    return np.array(out, dtype=np.float32)

Xtr, Xva, Xte = unify(Xtr_list), unify(Xva_list), unify(Xte_list)

# Scale features (fit on train only)
scaler = StandardScaler()

def fit_transform_3d(X, fit=False):
    sh = X.shape
    Z = X.reshape(sh[0], -1)
    Z = scaler.fit_transform(Z) if fit else scaler.transform(Z)
    return Z.reshape(sh).astype(np.float32)

Xtr = fit_transform_3d(Xtr, fit=True)
Xva = fit_transform_3d(Xva)
Xte = fit_transform_3d(Xte)

# Add channel dimension
Xtr = Xtr[..., None]
Xva = Xva[..., None]
Xte = Xte[..., None]

# Encode labels
le = LabelEncoder().fit(sorted(set(labels)))
ytr = le.transform(ytr_txt)
yva = le.transform(yva_txt)
yte = le.transform(yte_txt)
```

```
# Persist arrays and transformers
np.save(PROCESSED/'X_train.npy', Xtr)
np.save(PROCESSED/'y_train.npy', ytr)
np.save(PROCESSED/'X_val.npy', Xva)
np.save(PROCESSED/'y_val.npy', yva)
np.save(PROCESSED/'X_test.npy', Xte)
np.save(PROCESSED/'y_test.npy', yte)
with open(PROCESSED/'label_encoder.pkl', 'wb') as f:
    pickle.dump(le, f)
with open(PROCESSED/'scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

# Save meta for cache validation
meta = {
    'config': CONFIG,
    'classes': le.classes_.tolist(),
    'shapes': {
        'X_train': list(Xtr.shape), 'X_val': list(Xva.shape), 'X_test': list
    }
}
with open(META_JSON, 'w') as f:
    json.dump(meta, f)

log(f"Saved processed FMA arrays: {Xtr.shape}, {Xva.shape}, {Xte.shape}")
log(f"Class mapping: {dict(zip(le.classes_.tolist(), range(len(le.classes_))
```

```
[17:46:10] INFO: Usando metadata: /home/alepot55/Desktop/projects/naml_proje
ct/data/fma_download/fma_metadata/tracks.csv
[17:46:12] INFO: Applied cap per class: 40
[17:46:12] INFO: Classes: ['Electronic', 'Experimental', 'Folk', 'Hip-Hop',
'Instrumental', 'International', 'Pop', 'Rock'] | Total tracks considered: 3
20
[17:46:12] INFO: Stratify enabled: True
[17:46:12] INFO: Train/Val/Test sizes: 192/64/64
[17:46:12] INFO: Applied cap per class: 40
[17:46:12] INFO: Classes: ['Electronic', 'Experimental', 'Folk', 'Hip-Hop',
'Instrumental', 'International', 'Pop', 'Rock'] | Total tracks considered: 3
20
[17:46:12] INFO: Stratify enabled: True
[17:46:12] INFO: Train/Val/Test sizes: 192/64/64
100%|██████████| 192/192 [00:50<00:00,  3.79it/s]
100%|██████████| 192/192 [00:50<00:00,  3.79it/s]
100%|██████████| 64/64 [00:16<00:00,  3.78it/s]
100%|██████████| 64/64 [00:16<00:00,  3.78it/s]
100%|██████████| 64/64 [00:17<00:00,  3.73it/s]

[17:47:38] INFO: Saved processed FMA arrays: (1920, 128, 129, 1), (640, 128,
129, 1), (640, 128, 129, 1)
[17:47:38] INFO: Class mapping: {'Electronic': 0, 'Experimental': 1, 'Folk':
2, 'Hip-Hop': 3, 'Instrumental': 4, 'International': 5, 'Pop': 6, 'Rock': 7}
```

In [6]:

```
# Final quick summary and verification (fast; uses cache if present)
import numpy as np, pandas as pd, pickle
from collections import Counter

print('Processed arrays saved in:', PROCESSED)
```

```
print('X shapes:', np.load(PROCESSED/'X_train.npy', mmap_mode='r').shape, np

ytr = np.load(PROCESSED/'y_train.npy', mmap_mode='r'); yva = np.load(PROCESS
with open(PROCESSED/'label_encoder.pkl','rb') as f:
    le = pickle.load(f)

idx_to_name = {i: cls for i, cls in enumerate(le.classes_)}

def fmt_counts(arr):
    cnt = Counter(np.asarray(arr).astype(int).tolist())
    return {idx_to_name[k]: int(v) for k, v in sorted(cnt.items())}

print('Train counts:', fmt_counts(ytr))
print('Val counts:', fmt_counts(yva))
print('Test counts:', fmt_counts(yte))
```

```
Processed arrays saved in: /home/alepot55/Desktop/projects/naml_project/dat
a/processed_fma
X shapes: (1920, 128, 129, 1) (640, 128, 129, 1) (640, 128, 129, 1)
Train counts: {np.str_('Electronic'): 240, np.str_('Experimental'): 240, np.
str_('Folk'): 240, np.str_('Hip-Hop'): 240, np.str_('Instrumental'): 240, n
p.str_('International'): 240, np.str_('Pop'): 240, np.str_('Rock'): 240}
Val counts: {np.str_('Electronic'): 80, np.str_('Experimental'): 80, np.str_
('Folk'): 80, np.str_('Hip-Hop'): 80, np.str_('Instrumental'): 80, np.str_
('International'): 80, np.str_('Pop'): 80, np.str_('Rock'): 80}
Test counts: {np.str_('Electronic'): 80, np.str_('Experimental'): 80, np.str
_('Folk'): 80, np.str_('Hip-Hop'): 80, np.str_('Instrumental'): 80, np.str_
('International'): 80, np.str_('Pop'): 80, np.str_('Rock'): 80}
```

In [7]:
```
# Opzionale: anteprima metadati locale (tracks.csv) ed esempio KaggleHub dis
import pandas as pd
from pathlib import Path
import os
import sys

# Resolve PROJECT_ROOT if missing (in case cell 1 wasn't run)
try:
    PROJECT_ROOT
except NameError:
    from pathlib import Path
    PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]

# Try to use TRACKS_CSV from previous cell; otherwise, fall back to default
try:
    TRACKS_CSV
except NameError:
    METADATA_ROOT = PROJECT_ROOT/'data'/'fma_metadata'
    TRACKS_CSV = METADATA_ROOT/'tracks.csv'
    if not TRACKS_CSV.exists():
        dl_dir = PROJECT_ROOT/'data'/'fma_download'
        cand = list(dl_dir.rglob('tracks.csv')) if dl_dir.exists() else []
        if cand:
            TRACKS_CSV = cand[0]

print('tracks.csv path candidate:', TRACKS_CSV)
if Path(TRACKS_CSV).exists():
```

```
    df_tracks = pd.read_csv(TRACKS_CSV, header=[0,1], index_col=0, low_memor
    print('tracks.csv loaded. Shape:', df_tracks.shape)
    display(df_tracks.head())
else:
    print('tracks.csv non trovato. Esegui le prime celle per scaricare i met

# Facoltativo: esempio KaggleHub (disabilitato di default)
USE_KAGGLEHUB = False  # imposta a True solo se sai esattamente cosa stai ca
DATASET_ID = "mdeff/fma"   # dataset ufficiale; i CSV sono in fma_metadata.z
FILE_PATH = "tracks.csv"   # nome del file da leggere (se disponibile via ad

if USE_KAGGLEHUB:
    try:
        import kagglehub
        # Nota: kagglehub.load_dataset (Pandas adapter) è deprecato e può da
        # Percorso consigliato: scaricare l'archivio con kagglehub.dataset_d
        base_path = kagglehub.dataset_download(DATASET_ID)
        base_path = Path(base_path)
        print('KaggleHub cache:', base_path)
        # Se presente, prova a trovare tracks.csv o estrarre fma_metadata.zi
        cand = list(base_path.rglob('tracks.csv'))
        if not cand:
            zips = list(base_path.rglob('fma_metadata*.zip'))
            if zips:
                import zipfile
                with zipfile.ZipFile(zips[0], 'r') as zf:
                    zf.extractall(zips[0].parent)
                cand = list(base_path.rglob('tracks.csv'))
        if cand:
            df_kh = pd.read_csv(cand[0], header=[0,1], index_col=0, low_memo
            print('KaggleHub tracks.csv loaded. Shape:', df_kh.shape)
            display(df_kh.head())
        else:
            print('KaggleHub: tracks.csv non trovato nel dataset scaricato.'
    except Exception as e:
        print('KaggleHub preview failed:', e)
```

```
tracks.csv path candidate: /home/alepot55/Desktop/projects/naml_project/dat
a/fma_download/fma_metadata/tracks.csv
tracks.csv loaded. Shape: (106574, 52)
```

track_id	comments	date_created	date_released	engineer	favorites	id	information
2	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1	
3	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1	
5	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1	
10	0	2008-11-26 01:45:08	2008-02-06 00:00:00	NaN	4	6	NaN
20	0	2008-11-26 01:45:05	2009-01-06 00:00:00	NaN	2	4	"spiritual songs" from Nicky Cook

5 rows × 52 columns

In [8]:
```
# Quick diagnostics: check mp3 coverage and labels without extracting featur
from pathlib import Path
import os, pandas as pd

# Resolve roots
try:
    PROJECT_ROOT
except NameError:
    from pathlib import Path
    PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
FMA_ROOT = PROJECT_ROOT/'data'/'fma_small'
METADATA_ROOT = PROJECT_ROOT/'data'/'fma_metadata'
TRACKS_CSV = METADATA_ROOT/'tracks.csv'

print('FMA_ROOT exists:', FMA_ROOT.exists())
mp3_files = sorted([p for p in FMA_ROOT.rglob('*.mp3')]) if FMA_ROOT.exists(
print('MP3 files found:', len(mp3_files))

if not TRACKS_CSV.exists():
    # Try find tracks.csv in download cache
    dl_dir = PROJECT_ROOT/'data'/'fma_download'
    cand = list(dl_dir.rglob('tracks.csv')) if dl_dir.exists() else []
```

```
    if cand:
        TRACKS_CSV = cand[0]

print('TRACKS_CSV:', TRACKS_CSV, '| exists:', TRACKS_CSV.exists())

if TRACKS_CSV.exists() and mp3_files:
    tracks = pd.read_csv(TRACKS_CSV, header=[0,1], index_col=0, low_memory=F
    if ('track','genre_top') not in tracks.columns:
        raise RuntimeError('tracks.csv missing (track, genre_top) column')
    id_to_genre = tracks[('track','genre_top')]

    def parse_tid(p: Path):
        try:
            return int(p.stem)
        except Exception:
            return None

    tids = [parse_tid(p) for p in mp3_files]
    tids = [t for t in tids if t is not None]
    labeled = id_to_genre.loc[id_to_genre.index.intersection(tids)]
    non_na = labeled.dropna()

    print('Unique track ids in mp3s:', len(set(tids)))
    print('Labeled tracks with non-NaN genre_top:', non_na.shape[0])
    print('Top 10 genre_top counts among labeled:')
    print(non_na.value_counts().head(10))
else:
    print('Skipping label diagnostics: missing mp3s or tracks.csv')
```

```
FMA_ROOT exists: True
MP3 files found: 8000
TRACKS_CSV: /home/alepot55/Desktop/projects/naml_project/data/fma_download/f
ma_metadata/tracks.csv | exists: True
Unique track ids in mp3s: 8000
Labeled tracks with non-NaN genre_top: 8000
Top 10 genre_top counts among labeled:
(track, genre_top)
Hip-Hop          1000
Pop              1000
Folk             1000
Experimental     1000
Rock             1000
International    1000
Electronic       1000
Instrumental     1000
Name: count, dtype: int64
```